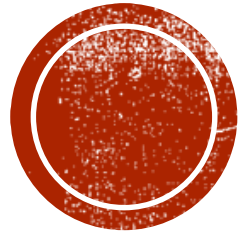


FUNCTION OVERLOADING





PRACTICE EXAMPLE

Absoulte.cpp

SCS1310

```

1  #include <iostream>
2  using namespace std;
3
4  int abs(int i);
5  double abs(double d);
6  long long abs(long long l);
7
8  int abs(int i){
9      cout<<"Integer"<<endl;
10     return i < 0 ? -i : i;
11 }
12
13 double abs(double d){
14     cout<<"Double"<<endl;
15     return d < 0.0 ? -d : d;
16 }
17
18 long long abs(long long l) {
19     cout<<"Long"<<endl;
20     return l < 0 ? -l : l;
21 }

```

```

23 int main(void)
24 {
25     cout << abs(-11) << endl;
26     cout << abs(-22.2) << endl;
27     cout << abs(-214748364756) << endl;
28 }

```

```

Integer
11
Double
22.2
Long
214748364756

```

```

-----
Process exited after 0.04974 seconds with return value 0
Press any key to continue . . .

```



WHAT IS FUNCTION OVERLOADING?

- The C language forces us to use different names for similar functions.

- Example:

```
int abs(int);  
double fabs(double);  
long labs(long);
```

- But C++ lets us use just one name for all similar functions.
 - Function overloading is a feature in C++
- Allows two or more functions to have the same name with different parameters.
- This is an example of polymorphism feature in C++.



FUNCTION OVERLOADING

- Defining two or more functions using the same name but different parameters.
- Overloaded functions must differ in their parameter lists
 - must have either different numbers of parameters or
 - different types of parameters.
- When an overloaded function is called, the **compiler uses the argument list** to determine which function of that name is being invoked

```
int abs(int i);  
double abs(double d);  
long abs(long l);
```



FUNCTION OVERLOADING

```
int getValue()  
{  
    return 1;  
}  
  
float getValue()  
{  
    return 1.2;  
}
```



Compilation Error



FUNCTION OVERLOADING & RETURN TYPES

- Functions can not be overloaded if they differ only in the return type.

```
int getValue(int i);  
float getValue(int i);
```

// This attempt to overload will fail



FUNCTION OVERLOADING

```
float getSquare(float f);  
double getSquare(double d);
```

```
//Function call  
cout << getSquare(10);
```



Compilation Error



AVOIDING AMBIGUITY

- When overloading functions, it is possible to create **ambiguous situations** where the compiler is unable to choose between two or more overloaded functions.
- If there are Ambiguous statements the compilations errors will occur.

Implicit type conversion

```
float getSquare(float f);  
double getSquare(double d);
```

```
//Function call  
cout << getSquare(10);
```



DEFAULT ARGUMENTS

- Default arguments are one possible cause of ambiguity.

```
void addPerson(char* firstName, char* lastName, int age = 0);  
  
void addPerson(char* firstName, char* lastName);
```

- Attempt to call the second version of **addPerson()** will be ambiguous.

```
addPerson("Jane", "Citizen");
```

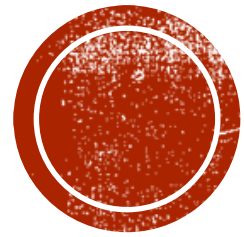


RESOLVING CALLS TO OVERLOADED FUNCTIONS

The compiler uses the following selection algorithm to help it make the best choice when resolving calls to overloaded functions:

1. Use an exact match if possible.
2. Try standard type promotions (e.g. **char** to **int**).
3. Try standard type conversions (e.g. **int** to **float**).
4. Try user-defined type conversions (e.g. conversion constructor or operator conversion function).





QUIZ—FUNCTION OVERLOADING

QUESTION 01

Which of the following in Object Oriented Programming is supported by Function overloading?

1. Inheritance
2. Run-time Polymorphism
3. Encapsulation
4. Compile-time Polymorphism
5. None of the above

**Answer: Compile-time
Polymorphism**



QUESTION 02

```
1  #include<iostream>
2  using namespace std;
3
4  void test(int s, int t, int x=0)
5  {
6      cout << "Function with three para called ";
7      cout<<s<<" "<<t<<" "<<x<<endl;
8  }
9
10 void test(int s, int t)
11 {
12     cout << "Function with two para called \n";
13     cout<<s<<" "<<t<<endl;
14 }
15
16
17 int main()
18 {
19     test(4,5);
20     test(4,5,67);
21 }
```



QUESTION 03

Answer: All

Which of the following overloaded functions are NOT allowed in C++?

1. Function declarations that differ only in the return type
2. Functions that differ only by static keyword in return type
3. Parameter declarations that differ only in a pointer * versus an array []
4. Two parameter declarations that differ only in their default arguments

```
int fun(int x, int y);  
void fun(int x, int y);
```

```
int fun(int x, int y);  
static int fun(int x, int y);
```

```
int fun(int *ptr, int n);  
int fun(int ptr[], int n);
```

```
int fun( int x, int y);  
int fun( int x, int y = 10);
```



QUESTION 04

```
1  #include <iostream>
2  using namespace std;
3
4  void display(char c) {
5      cout << "Character: " << c << endl;
6  }
7
8  void display(float a) {
9      cout << "Float: " << a << endl;
10 }
11
12 void display(char c, int a) {
13     cout << "Character and Integer: " << c << ", " << a << endl;
14 }
15
16 void display(double a, int b) {
17     cout << "Double and Integer: " << a << ", " << b << endl;
18 }
```

```
20 int main() {
21     display(50);
22     display('A');
23     display(3.14);
24     display('B', 10);
25     display(20.5, 5);
26     return 0;
27 }
```



HOME WORK

Write a program that calculates the area of different geometric shapes (circle, rectangle, and triangle) using function overloading.

Select a shape to calculate the area:

1. Circle
2. Rectangle
3. Triangle

Enter your choice: 1

Enter the radius of the circle: 5

Area of the circle: 78.54

Select a shape to calculate the area:

1. Circle
2. Rectangle
3. Triangle

Enter your choice: 2

Enter the length and breadth of the rectangle: 4 6

Area of the rectangle: 24.00

Select a shape to calculate the area:

1. Circle
2. Rectangle
3. Triangle

Enter your choice: 3

Enter the base and height of the triangle: 5 7

Area of the triangle: 17.50



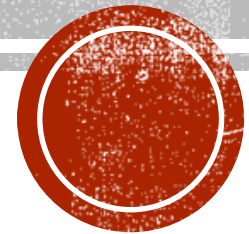
FUNCTION OVERLOADING IN SUMMARY

- Define multiple definitions for the same function name in the same scope.
- The definition of the function must differ from each other by the types and/or the number of arguments in the argument list.
- Cannot overload function declarations that differ only by return type.

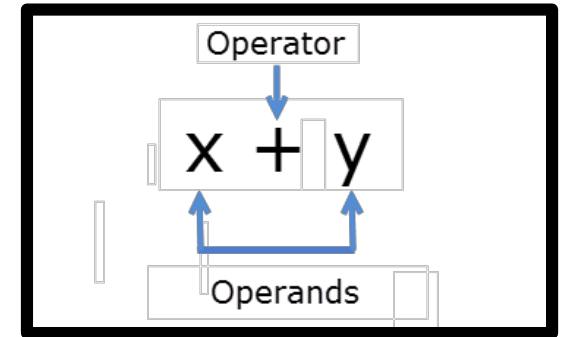


OPERATOR OVERLOADING

What is meant by Operator Overloading?



WHY—OPERATOR OVERLOADING



- Operators work with operands.
- Increment operator can be used with int values or int type variables.
Example: count ++.

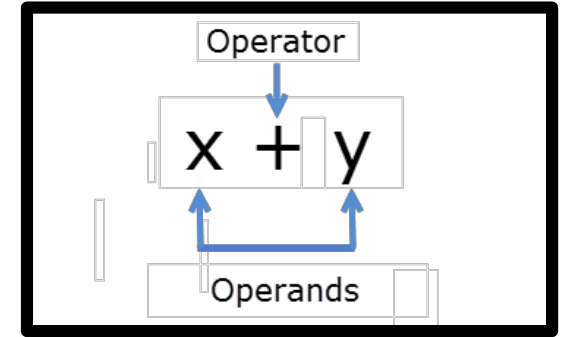
21

Process exited after 0.07273 seconds with return value 0
Press any key to continue . . . |

```
1  #include<iostream>
2  using namespace std;
3
4  int main(){
5
6      int i, j;
7
8          j = 10;
9          i = (j++, j+100, j+10);
10
11         cout << i;
12
13         return 0;
14 }
```



WHY—OPERATOR OVERLOADING



- Operators work with operands.
- Increment operator can be used with int values or int type variables.
Example: count ++.

Question: Can we use the above operator with objects directly?

Example: Create an object (N1) from Numbers class and increment all the variables inside the class?

We can't use operators on objects as they are NOT applied for classes.

However, these operators can be overloaded to get special functions done.

```
class Numbers{  
    int x, y;  
  
    public:  
    Numbers():x(0),y(0){  
    }  
};
```

```
int main(){  
    Numbers N1;  
    ++N1;  
}
```



EXERCISE

My Name

Process exited after 0.2586 seconds with return value 0
Press any key to continue . . . |

```
1  #include<iostream>
2  using namespace std;
3
4  int main()
5  {
6      string x="My ";
7      string y="Name";
8
9      cout<<endl<<x+y;
10 }
```



WHAT IS OPERATOR OVERLOADING

- In C++, we can make operators to work for user defined classes.
- Allows to change the use of operators such as +, -, ++, & etc.
- Applied for user defined classes to create the own code to give them new functionality.

Example 1: Use '+' operator to concatenate two strings together

`str1 = str2 + str3`, without calling `append()` function.

Example 2: Override '++' operator not to increment value by 1, but, by 100.



HOW TO DEFINE OPERATOR OVERLOADING

- Operators are overloaded by defining an operator function.
 - An operator function defines the specific operations that the overloaded operator will perform relative to the class it is designed to work on.
- We must use keyword “operator” before the operator we want to overload
- Works as a function therefore the return type must be defined.

Syntax: operator function

```
returnType operator operatorSymbol(parameter)
{
    // ...
}
```

- **BEST PRACTISE:** Although you are free to perform any operation you want inside an operator function it is usually best to stay within the context of the operator's normal use.



DIFFERENCE BETWEEN OPERATOR FUNCTIONS AND NORMAL FUNCTIONS

SIMILARITY:

- Operator functions are same as normal functions.

DIFFERENCE:

- Name of an operator function is the operator keyword followed by symbol of operator
- Operator functions are called when the corresponding operator is used.

Operator Function: `Nl++;`

Normal Function: `Nl.numIncrement();`



CAN WE OVERLOAD ALL OPERATORS?

NO.

Operators that can NOT be Overloaded:

- The dot operator (.),
- The scope resolution operator (::),
- The dereferencing operator (*),
- The ternary conditional operator (? :) and
- The **sizeof** operator

WHY? (H/W)



RESTRICTIONS ON OPERATOR FUNCTIONS

- New operators cannot be created.
- Only existing built-in operators can be overloaded.
- An operator's precedence cannot be changed.
- An operator's associativity cannot be altered.
- Cannot change the number of operands that an operator takes.
- Operator functions cannot have default arguments.
- Some operators are by default overloaded.
 - Assignment (=)
 - Address (&)
 - Comma (,)



TYPES OF OPERATOR OVERLOADING

There are **three approaches**

- Overloading unary operator.
- Overloading binary operator.
- Overloading operators using a friend function.

What is a unary operator?

- Operators act on only one operand.
- Examples: increment (++), decrement (--), minus (-) : -33.

What is a binary operator?

- Operators act on two operands.
- Examples: +, -, <=, >, ==,



OVERLOADING UNARY OPERATOR

- No arguments should be passed.
- Works only with one class objects.
- Overloading an operator operating on a single operand.



OVERLOADING UNARY OPERATOR: EXAMPLE 1

```
class Number{
    int num;

    public:
    Number(int a):num(a){
    }

    void display(){
        cout<<"Number variable: "<<num<<endl;
    }

    void operator -(){
        this->num=-num;
    }

};
```

```
int main()
{
    Number myNum1(6);
    -myNum1;

    myNum1.display();
}
```



OVERLOADING UNARY OPERATOR: EXAMPLE 2

```
class Counter{
    unsigned int count;

public:
    Counter():count(0) //constructor
    { }

    void operator++(){
        ++count;
    }

    int getCounter()
    {
        return count;
    }
};
```

Prefix Increment

```
int main()
{
    Counter c1;
    ++c1;           //Prefix

    cout<<"Count= "<<c1.getCounter()<<endl;
}
```



OVERLOADING UNARY OPERATOR: EXAMPLE 3

Postfix Increment

The variable is incremented after its value is used in the expression

Not an Argument



```
Counter operator++(int) {    //postfix increment
    return Counter(count++);
}
```



THE INCREMENT OPERATOR OVERLOADING

The general forms of the prefix and postfix **operator++()** functions

```
// Prefix increment operator ++ObjectName  
returnType ClassName::operator++()  
{  
    // ...  
}
```

```
// Postfix increment operator ObjectName++  
returnType ClassName::operator++(int)  
{  
    // ...  
}
```



THE DECREMENT OPERATOR OVERLOADING

The general forms of the prefix and postfix **operator--()** functions

```
// Prefix decrement operator --ObjectName  
returnType ClassName::operator--()  
{  
    // ...  
}
```

```
// Postfix decrement operator ObjectName--  
returnType ClassName::operator--(int)  
{  
    // ...  
}
```



EXERCISE

```
1  #include<iostream>
2  using namespace std;
3
4  class Number{
5      int x,y;
6
7      public:
8          Number():x(0),y(0){}
9
10         void operator++(){
11             x=x+100;
12             y=y+100;
13         }
14
15         void display(){
16             cout<<"X is "<<x;
17             cout<<"\nY is "<<y<<endl;
18         }
19     };
20
```

```
21  int main(){
22
23      Number N1,N2;
24      N1.display();
25
26      ++N1;
27      N1.display();
28      N2.display();
29  }
30
```

```
X is 0
Y is 0
X is 100
Y is 100
X is 0
Y is 0
```

```
-----
Process exited after 1.042 seconds with return value 0
Press any key to continue . . . |
```



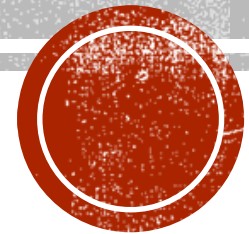
OVERLOADING BINARY OPERATOR

- Overloading of an operator operating on two operands.
- One argument should be passed to the operator function.



PRACTICE EXERCISE

Complex Number Addition & Subtraction



```

#include<iostream>
using namespace std;

class Complex{
    int real;
    int imagi;

public:
    Complex():real(0),imagi(0){

        cout<<"Inside the constructor"<<endl;}

    Complex(int x, int y):real(x),imagi(y){}

    Complex(Complex& c){

        cout<<"User Defined copy constructor"<<endl;
        real=c.real;
        imagi=c.imagi;
    }

    Complex operator +(Complex& c){
        Complex tempc;
        tempc.real=real+c.real;
        tempc.imagi=imagi+c.imagi;
        return (tempc);
    }
}

```

```

Complex operator -(Complex& c){
    Complex tempc;
    tempc.real=real-c.real;
    tempc.imagi=imagi-c.imagi;
    cout<<c.real<<" "<<c.imagi<<endl;
    return (tempc);
}

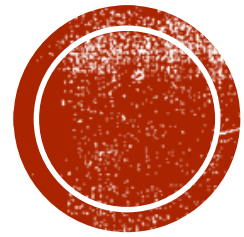
void display(){
    if(imagi < 0)
        cout << "Complex No: " << real << imagi << "i"<<endl;
    else
        cout << "Complex No: " << real << "+" << imagi << "i"<<endl; }

~Complex()
{
    cout<<"Deallocate Memory"<<endl;
}

};

int main(){
    Complex C1(3,4);
    C1.display();
    Complex C2(2,3);
    C2.display();
    C1=C2-C1;
    C1.display();
    Complex C3=C1;
    C3.display();
}

```



RECAP

Operator Overloading

SCS1310

DIFFERENT TYPES

- Overloading unary operator.
- Overloading binary operator.
- Overloading operators using a friend function.

A unary operator?

- Operators act on only one operand.
- Examples: increment (++), decrement (--), minus (-) : -33.

A binary operator?

- Operators act on two operands.
- Examples: +, -, <=, >, ==,



```

#include<iostream>
using namespace std;

class Complex{
    int real;
    int imagi;

public:
    Complex():real(0),imagi(0){

        cout<<"Inside the constructor"<<endl;}

    Complex(int x, int y):real(x),imagi(y){}

    Complex(Complex& c){

        cout<<"User Defined copy constructor"<<endl;
        real=c.real;
        imagi=c.imagi;
    }

    Complex operator +(Complex& c){
        Complex tempc;
        tempc.real=real+c.real;
        tempc.imagi=imagi+c.imagi;
        return (tempc);
    }
}

```

```

Complex operator -(Complex& c){
    Complex tempc;
    tempc.real=real-c.real;
    tempc.imagi=imagi-c.imagi;
    cout<<c.real<<" "<<c.imagi<<endl;
    return (tempc);
}

void display(){
    if(imagi < 0)
        cout << "Complex No: " << real << imagi << "i"<<endl;
    else
        cout << "Complex No: " << real << "+" << imagi << "i"<<endl; }

~Complex()
{
    cout<<"Deallocate Memory"<<endl;
}

};

int main(){
    Complex C1(3,4);
    C1.display();
    Complex C2(2,3);
    C2.display();
    C1=C2-C1;
    C1.display();
    Complex C3=C1;
    C3.display();
}

```

```
Complex& operator =(const Complex &c){  
    real=c.real;  
    imagi=c.imagi;  
    cout<<"Inside the assignment operator"<<endl;  
    return *this;  
}
```



OVERLOADED ASSIGNMENT OPERATOR

```
ClassName& ClassName::operator=(const ClassName& op)
{
    //.....
    return *this;
}
```



ASSIGNMENT OPERATOR

- Compiler by default automatically defines an operator overloading for the assignment operator.
- Used to **copy values** from one object to another *already existing object*.

Example: obj1 and obj2

obj1 = obj2 (Copy all the data values of obj2 into obj1)

- Only shallow copy (makes a bitwise copy of the class instance) is done hence may cause problem while working with pointers.



ASSIGNMENT VS COPY CONSTRUCTOR

Feature	Copy Constructor	Assignment Operator (operator=)
Purpose	Creates a new object as a copy of an existing object	Copies the content of one existing object to another existing object
When called	During initialization of a new object	When assigning an already created object
Signature	ClassName(const ClassName &obj)	ClassName& operator=(const ClassName &obj)
Memory	Allocates new memory for the object	No new memory allocated; updates existing object
Example line	ClassName obj2 = obj1; or ClassName obj2(obj1);	obj2 = obj1;



ACTIVITY: WRITE THE OUTPUT

```
class Test {  
    public:  
        Test() {cout << "Constructor\n";}  
        Test(const Test& t) { cout << "Copy Constructor\n";}  
        Test& operator=(const Test& t) {  
            cout << "Assignment Operator\n";  
            return *this;  
        }  
};  
  
int main() {  
    Test a;  
    Test b = a;  
    Test c;  
    c = a;  
}
```



ACTIVITY

Which of the following statements invokes the assignment operator? Justify in one sentence

- A. `MyClass obj1(obj2);`
- B. `MyClass obj1 = obj2;`
- C. `obj1 = obj2;`
- D. `MyClass obj1;`



ACTIVITY

Which of the following statements invokes the assignment operator? Justify in one sentence

A. MyClass obj1(obj2);

B. MyClass obj1 = obj2;

C. obj1 = obj2;

D. MyClass obj1;

Answer: C

Justification:

The assignment operator is invoked when copying values into an object that has already been created.

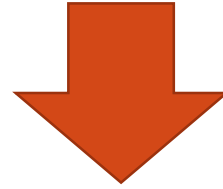


OPERATOR FUNCTIONS: MEMBER/NON-MEMBER

- Operator functions can be either defined as members or non-members of the class
- Non-member operator functions
 - Should be defined as friend functions
 - Need access to the class's private data members.
- The way an operator function is written differs depending on whether it is a member function or a friend function.



A fundamental concept of object-oriented programming



DATA HIDING

- **Restricts access to private members from outside the class.**
- **Protected members can only be accessed by derived classes and are inaccessible from outside.**



FRIEND FUNCTIONS

- What is a friend function?
 - A friend function is a function that is not a member of a class but is granted access to its private and protected members.
- Why use a friend function?
 - Allow an external function (or another class) to access private/protected data of a class without making the data public.
 - Useful for operator overloading and functions that work on multiple classes.
- How do you define and declare a friend function?
 - **Declare** the function **inside** the class with the friend keyword.
 - **Define** it **outside** the class, similar to a regular function.



FRIEND OPERATOR FUNCTIONS

Binary operator overloaded using a friend function:

- The **left operand is passed in as the first parameter** and the **right operand is passed in as the second parameter**.

```
friend Complex& operator+(const Complex& ob1, const Complex& ob2);
```

Unary operator overloaded using a friend function:

- the operand is passed to the function as a parameter.

```
friend Counter& operator++(Counter& ob);
```

It is common for an overloaded operator function's return type to be the same as the class it operates on, so that several operations may be chained together.

An operator function that changes the value of its invoking operand normally returns a reference to that operand.



MEMBER OPERATOR FUNCTIONS

Binary operator overloaded using a member function:

- The left operand generates the function call, and the right operand is passed to the function as a parameter.

Unary operator overloaded using a member function:

- The operand generates the function call, parameter is not required.



THE ASSIGNMENT OPERATOR

- The assignment operator must be overloaded as a member function, not a friend function

[Justify (H/W)]

- The left operand invokes the **operator=()** function and accepts a constant reference to the right operand as an argument.



THE OUTPUT OPERATOR

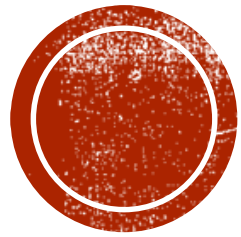
- The output operator **must** be overloaded as a friend function because its left operand is an output stream such as **cout**.
- The general form of the **operator<<()** function is
ostream& operator<<(ostream& out, const ClassName& op)
{
// ...
}
- The **operator<<()** function returns a reference to an **ostream** object so that output operations may be chained together.



THE INPUT OPERATOR

- The input operator must be overloaded as a friend function because its left operand is an input stream such as **cin**.
- The general form of the **operator>>()** function is
istream& operator>>(istream& in, ClassName& op)
{
 // ...
}
- The **operator>>()** function returns a reference to an **istream** object so that input operations may be chained together.





ACTIVITY

Input/Output Operator Overloading

STEP 1

Create a Complex class with two private members (real and imagi) and write a default constructor to initialize both variables to 0.

```
class Complex {  
    int real, imagi;  
    public:  
        Complex(int r=0, int i=0) : real(r), imagi(i) {}  
};
```



STEP 2

Write a left shift operator overloading function so that when you type `cout << c`, to print `a+bi` or `a-bi`.

```
friend ostream& operator<<(ostream& out, const Complex& c);  
  
ostream& operator<<(ostream& out, const Complex& c)  
{  
    if (c.imagi < 0)  
        out << c.real << c.imagi << "i";  
    else  
        out << c.real << "+" << c.imagi << "i";  
    return out;  
}
```



STEP 3

Write a right shift operator overloading function to read real and imaginary parts, when you type `cin >> c1;`

```
friend istream& operator>>(istream& in, Complex& c);

istream& operator>>(istream& in, Complex& c)
{
    cout << "Enter real part: ";
    in >> c.real;
    cout << "Enter imaginary part: ";
    in >> c.imagi;
    return in;
}
```



STEP 4

Create three objects and use chained input/output.

```
int main() {  
    Complex c1, c2, c3;  
  
    cin >> c1 >> c2 >> c3; // Chained input  
    cout << "\nYou entered:\n";  
    cout << c1 << " " << c2 << " " << c3 << endl; // Chained output  
    return 0;  
}
```



STEP 5

Write the Output

Enter real part: 2

Enter imaginary part: 3

Enter real part: -1

Enter imaginary part: 4

Enter real part: 0

Enter imaginary part: -5

You entered:

$2+3i$ $-1+4i$ $0-5i$

```

#include <iostream>
using namespace std;

class Fraction {
private:
    int numerator;
    int denominator;

public:
    // constructor
    Fraction(int x = 0, int y = 1) {
        numerator = x;
        denominator = y;
    }

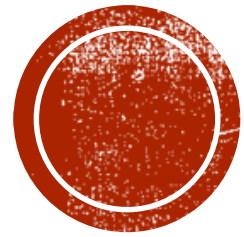
    friend istream& operator>>(istream& cin, Fraction& c){
        cin >> c.numerator >> c.denominator;
        return cin;
    }

    friend ostream& operator<<(ostream&, Fraction& c) {
        cout << c.numerator << "/" << c.denominator;
        return cout;
    }
};

int main()
{
    Fraction x;
    cout << "Enter a numerator and denominator of "
           "Fraction: ";
    cin >> x;
    cout << "Fraction is: ";
    cout << x;
    return 0;
}

```





DATA CONVERSION

DATA CONVERSION

- Assignments between the same data types are handled by the compiler.
 - Basic types / user-defined types

but....

Example:

a=b;

myObj1=myObj2;

- What happens when the variables on different sides of the = are of different types?
- Basic types – Compiler handles automatically
- Compiler automatically does not handle
 - Conversions between basic types and user-defined types, and
 - Conversions between different user-defined types.



CONVERSIONS BETWEEN BASIC TYPES

- Basic Data Types
 - Float to int
 - Float to double
 - Char to int
 - int to Char etc....
- Convert the value of floatvar (expressed in floating-point format) to an integer format and assigned to intvar.

```
intvar = floatvar;
```



DATA TYPE CONVERSION

- A type conversion occurs when the compiler changes the type of an object.
- An explicit type conversion occurs when the programmer uses a cast in the code to change the type of an object.

```
double x = double(3) / 2;    // The value of x is 1.5
```

- An implicit type conversion occurs when the compiler automatically changes the type of an object.

```
int i = 32.4;                // The value of i is 32
```



EXERCISE

```
1 #include<iostream>
2 using namespace std;
3
4 int main()
5 {
6     char a;
7     float x=100.9;
8
9     a = x;
10
11     cout<<"a: "<<a<<" "<<"x:"<<x<<" "<<(int)a<<endl;
12 }
```

```
a: d x:100.9 100
```

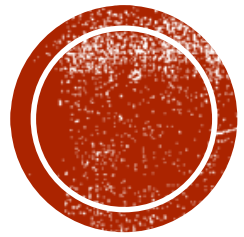
```
-----
Process exited after 0.1547 seconds with return value 0
Press any key to continue . . .
```



TYPE CASTING

- Converting a variable from one data type to another.
- **Implicit conversion**
 - Do not require any operator.
 - Automatically performed when a value is copied to a compatible type.
- **Explicit conversion**
 - There are many conversions
 - Imply a different interpretation of the value, require an explicit conversion.
- Casting operators:
 - `dynamic_cast`,
 - `reinterpret_cast`,
 - `static_cast` and
 - `const_cast`.

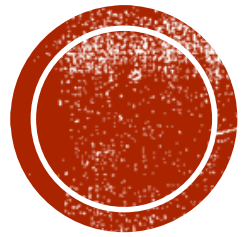




IS IT POSSIBLE?

Conversions between basic types and user-defined types ...

Conversions between different user-defined types ...



DATA CONVERSION

Conversion of primitive data type to user-defined

```
// Time Class
class Time {
    int hour;
    int mins;

public:
    // Default Constructor
    Time() : hour(0),mins(0){ }

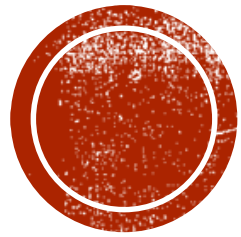
    // Parameterized Constructor
    Time(int t)
    {
        hour = t / 60;
        mins = t % 60;
    }

    void Display()
    {
        cout << "Time = " << hour
              << " hrs and "
              << mins << " mins\n";
    }
};
```

```
int main()
{
    // Object of Time class
    Time T1;
    int dur = 95;

    // Conversion of int type to class type
    T1 = dur;
    T1.Display();
}
```





DATA CONVERSION

Conversion of class object to primitive data type

CONVERSION OF CLASS OBJECT TO PRIMITIVE DATA TYPE

- Convert from class object and a primitive data type.
- Need to use an overloaded casting operator function (conversion function)

The syntax:

```
operator typename()
{
    // Code
}
```

Example:

The **operator double()** converts a class object to type double,

The **operator int()** converts a class type object to type int etc..



```

class Time {
    int hrs, mins;

public:
    // Constructor
    Time(int, int);

    // Casting operator
    operator int();

};

Time::Time(int a, int b)
{
    hrs = a;
    mins = b;
}

```

```

// Used for Data conversion of class to primitive
Time::operator int()
{
    cout << "Conversion of Class"
        << " Type to Primitive Type"
        << endl;

    return (hrs * 60 + mins);
}

int main()
{
    int hour, mins;
    hour = 3;
    mins = 20;

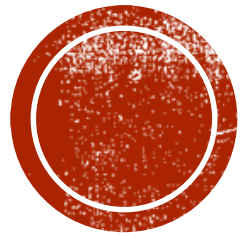
    Time t(hour, mins);

    int duration = t;

    cout << "Total Minutes are "
        << duration << endl;
}

```





DATA CONVERSION

Conversion of one class type to another class type

```
// Time Class
class Time {
    int hr, mins;
public:
    // Constructors
    Time(int h, int m) : hr(h),mins(m){ }
    Time()
    {
        cout << "\nTime's Object Created";
    }

    int Minute()
    {
        int totalmin = (hr * 60) + mins;
        return totalmin;
    }

    void show()
    {
        cout << "Hour: " << hr << endl;
        cout << "Minute : " << mins << endl;
    }
};
```

```
// Minutes Class
class Minute {
    int mins;
public:
    // Constructors
    Minute()
    {
        mins = 0;
    }
    void operator=(Time T)
    {
        mins = T.Minute();
    }

    void show()
    {
        cout << "\nTotal Minute : "
        << mins << endl;
    }
};
```

```
int main()
{
    int hour, mins;
    hour = 3, mins = 40;

    Time T1(hour,mins);
    T1.show();

    Minute M1;
    M1.show();
    M1=T1;
    M1.show();
}
```

